

Computer Systems Security

Lab-1

Wireshark Tool, Traffic Analysis Traces and Security Analysis of Traffic Captures

Lab Purpose

The purpose of this initial laboratory exercise is to download, install and use the **Wireshark** network protocol analyzer, analyzing previously captured network traffic traces.

Steps:

- Install Wireshark on **Windows, macOS, or Linux**.
- Open and examine packet-capture files supplied by the instructor.
- Identify network protocols and communication flows
- Remember the TCP/IP Stack to inspect packet headers and protocol fields and vulnerabilities at different levels
- Identify critical information transmitted in plaintext
- Discuss evidence-based conclusions on insecure communication from packet captures
- Investigate ARP traffic and identify evidence of ARP spoofing attacks.

No live packet capture is required for this exercise. All network traffic has already been captured and will be supplied as .pcap. See the pcap files referred for the Lab

First step: Install Wireshark and confirm the installation (if you don't have it)

Need:

- To use computers running **Windows, macOS, or Linux**.
- To download and install Wireshark
- The packet-capture files supplied by the instructor..
- A web browser or PDF reader for the laboratory instructions.

The official Wireshark website provides the current releases and installers for the above computers and Operating Systems

<https://www.wireshark.org/>

<https://www.wireshark.org/download.html>

For Linux users and depending on different distributions, can also install directly from command line, with appropriate permissions for the installation. After the installation, simply verify running `wireshark --version`

Ubuntu, Debian	<code>sudo apt update</code> <code>sudo apt install wireshark</code>
Fedora/RHEL	<code>sudo dnf install wireshark</code>
Arch	<code>sudo pacman -S wireshark-qt</code>

Initially for this Lab you don't need many complex features for the use of Wireshark. Later on you can explore the tool for use in different situations. Anyway, a User's Guide is available here:

https://www.wireshark.org/docs/wsug_html_chunked/

Confirm your Installation

After installing Wireshark, open the application. The Wireshark welcome screen should be displayed. Students don't need to **start initially a live packet capture** for the rest of this initial Lab but can try to see how to do this immediately. Instead, the next objective is to open the capture files supplied for the lab (as pre-captured traces in *pcap* files).

Wireshark can save observed network traffic captures to both *.pcap* and *.pcapng* files and read captured traffic from these file formats.

- **pcap** is a classic, flat-file format for single-interface packet captures
 - Uses a fixed sequential structure with a single global header and flat packet records.
 - Limited format for one link-layer type (e.g., only Ethernet or only Wi-Fi or only loopback interfaces for localhost traffic) per file from a single capture interface
 - Timestamp precision of the capture: microseconds
 - Stores basic packet data (on different protocols), length, and timestamps.
 - Universally supported by almost all legacy and modern network tools.
- **pcapng** (PCAP Next Generation) is a modern, block-based format that supports multiple interfaces, rich metadata, and advanced features **After the installation, simply verify running wireshark --version**
 - Uses a flexible, modular, block-based design that can easily expand over time
 - Supports simultaneous captures from multiple interfaces and mixed link-layer types in a single file
 - Timestamp precision: nanoseconds
 - Can embed more features, ex: user comments, more meta-data in certain protocols, name resolution data, capture statistics, etc.
 - The default format for other modern traffic analysis tools

Later on, you can see more about these formats in:

https://www.wireshark.org/docs/wsug_html_chunked/ChIOOpenSection.html#ChIOInputFormatsSection

Some initial useful filters for observable traffic

Basic protocol filters

telnet, ftp, ftp-data, http, dns, tcp, udp, http, tls

More for HTTP filtering:

```
http.request
http.response
http.request.method == "GET"
http.request.method == "POST"
http.authorization
```

More for IP (using address filtering)

```
ip.addr == <IP_ADDRESS>
ip.src == <IP_ADDRESS>
ip.dst == <IP_ADDRESS>
```

IP Communication between two hosts, ex:

```
ip.addr == <CLIENT_IP> && ip.addr == <SERVER_IP>
```

TCP (Filtering by ports)

```
tcp.port == 80  
tcp.port == 443
```

Conclusions from traffic analysis with Wireshark

More for ARP (requests/responses)

```
arp.opcode == 1  
arp.opcode == 2
```

More for TLS (handshakes)

```
tls.handshake  
tls.handshake.type == 1  
tls.handshake.type == 2  
tls.app_data
```

Part 1 – Traffic analysis for the given captured files

Open the given pcap files

For every capture file, students should first perform a general inspection to see the following

- Number of packets.
- Capture durations, where available.
- Protocols observed.
- Source addresses.
- Destination addresses.
- Application-level protocols and packet formats
- TCP/UDP transport packets and formats
- Possible obvious errors or unusual traffic signaling
- For any protocol can see packet sequences, the packet list, packet details and contents, or packet bytes

Initial traffic inspections

For each provided capture, identify:

1. Which hosts participated?
2. Which TCP or UDP ports are used?
3. What type of application data can be observed?
4. Can inspect traffic flows during captured sessions be reconstructed?
5. For the captured files and protocols, can you identify that the communication is provided by secure channel properties (considering authentication, confidentiality, integrity, replaying-protection), in information identified in the traffic analysis?
6. What security problem is demonstrated by the captures for the observed protocols?

For the above aspects, observe different traces.

1.1 For telnet traffic

1. Which hosts participated in the Telnet session?
2. Which TCP port was used?
3. Can application data be observed?
4. Can commands entered during the session be reconstructed?
5. Can authentication information be identified?
6. What security problem is demonstrated by the capture?

1.2 For ftp traffic

1. Inspect Client IP address.
2. Inspect Server IP address.
3. Inspect FTP control port.
4. Inspect the FTP Authentication exchange.
5. Inspect FTP commands and responses
6. Whether information is transmitted in plaintext.

1.3 For http traffic

1. Inspect requested URLs
2. Inspected traced HTTP methods (GET, POST, HEAD, PUT, etc.)
3. Inspect Host header.
4. Inspect IP Source and Destination addresses and ports
5. See HTTP response status.
6. See HTTP headers.
7. See visible application data

1.4 For http basic authentication traffic

1. Inspect the HTTP Authorization field.
2. What authentication mechanism is being used?
3. What information is present in the Authorization header?
4. Is the communication encrypted?
5. What security implications follow from using Basic Authentication over unencrypted HTTP?

Note; you have a small python program implementing a simple server using Basic HTTP Authentication you can test with your browser

Part 2 -. Observing an ARP Spoofing Trace

Remember initially what is a ARP Spoofing attack and how an ARP Spoof attack can be exploited and why.

Identify the captured traffic in ARP Spoofing trace

Identify Evidence of ARP Spoofing

Inspect the ARP packets and determine whether the same IP address appears to be associated with different MAC addresses.

Inspect

- Sender MAC address.

- Sender IP address.
- Target MAC address.
- Target IP address.

Students should identify the packet(s) that provide evidence of the ARP poisoning activity.

Part 3 . Conclusions and security analysis questions

Students must understand from possible traffic captures the notions related to the following questions

- What information is exposed by Telnet
- What information is exposed by FTP.
- What information is exposed by HTTP.
- The security implications of HTTP Basic Authentication without TLS.
- What information remains visible when TLS is used.
- What is an ARP Spoofing attack and how ARP spoofing affects local network communication?
- How ARP spoofing can be detected from packet captures.
- How can we prevent ARP Spoofing attacks?
 - See this: <https://www.okta.com/identity-101/arp-poisoning/>
 - Remember (from lectures) the protocols in TCP/IP security stack
- From the given traces, what protocols provide communication security properties, including: peer or message authentication, message-integrity, message confidentiality, message-replaying protection, as fundamental properties for secure channels?
- What is the difference between: peer-authentication and message authentication?
- What is the difference between connection-oriented confidentiality vs. connectionless confidentiality
- What is the difference between connection-oriented integrity vs. connectionless integrity
- What is the difference between partial vs. full message confidentiality?
- What is the difference between partial vs. full message integrity?
- What is the difference between message-confidentiality vs. traffic-flow confidentiality
- What is the difference between message-integrity vs. traffic-flow integrity
- What of the above properties you feel as desirable properties for secure communication channels for anonymous communication in a privacy-enhanced communication approach?

Complementary questions:

- What information is protected by TLS.
- Considering TLS in supporting HTTPS based web communication, what security properties (from the following) you can expect to have as guaranteed?
 - Peer-Authentication
 - Message Authentication
 - Message confidentiality
 - Message Integrity
 - Message replaying protection
 - Traffic-flow confidentiality
 - Traffic-flow integrity
 - Traffic Anonymization